Blocking My Crawl

Cite as: Martin Paul Eve, "Blocking My Crawl", *eve.gd: Martin Paul Eve*, November 09, 2023, https://doi.org/10.59348/eamzr-45367.

My day job involves quite a lot of crawling lists of websites to determine statistics about Crossref members and their behaviours. A good example is something I wanted to know recently: in the current sample, how many members display the title and doi (according to the latest display guidelines) on that page? In other words, how many members are doing good things on their landing pages? This is also important because, if a high number of these pages *are* behaving well, then we can use this as a marker of semantic shift/change. That is, we can use it to detect when a new domain owner comes along, for instance, and changes the content so that it no longer reflects the original.

The problem is, loads of scholarly publishers use DRM techniques on their sites that block my crawls. We use systems like Playwright to remote control browsers to do the crawling, so that the request looks as * much like a genuine user as possible. However, lots of these sites detect headless browsers and block them with a 403 Permission Denied error.

There's a great Github javascript suite that aims to help you evade headless detection. The tests it uses are:

- User Agent: in a browser running with puppeteer in headless mode, user agent includes Headless.
- App Version: same as User Agent above.

- Plugins: headless browsers don't have any plugins. So we can say that if it has plugin it's headful, but not otherwise since some browsers, like Firefox, don't have default plugins.
- Plugins Prototype: check if the Plugin and PluginsArray prototype are correct.
- Mime Type: similar to Plugins test, where headless browsers don't have any mime type
- Mime Type Prototype: check if the MimeType and MimeTypeArrayprototype are correct.
- Languages: all headful browser has at least one language. So we can say that if it has no language it's headless.
- Webdriver: this property is true when running in a headless browser.
- Time elapse: it pops an alert() on page and if it's closed too fast, means that it's headless.
- Chrome element: it's specific for chrome browser that has an element window.chrome.
- Permission: in headless mode Notification.permission and navigator.permissions.query report contradictory values.
- Devtool: puppeteer works on devtools protocol, this test checks if devtool is present or not.
- Broken Image: all browser has a default nonzero broken image size, and this may not happen on a headless browser.
- Outer Dimension: the attributes outerHeight and outerWidth have value 0 on headless browser.
- Connection Rtt: The attribute navigator.connection.rtt,if present, has value 0 on headless browser.
- Mouse Move: The attributes movementX and movementY on every MouseEvent have value 0 on headless browser.

Using the stealth plugin for Playwright allows me to evade most of these. I have also put in a pull request that patches the Connection Rtt value. This just leaves Mouse Move and Broken Image detection, which I thought would not outweigh all the other factors. I also jitter the connection with arbitrary delays so that it should appear to be coming at random intervals, rather than a robotic crawl.

Yet the basic fact is that I am still getting blocked. This does *not* happen when I put the browser into headful mode, so current detection techniques have clearly evolved in the past half decade (since Detect Headless) was designed. If anyone has further ideas or resources on headless detection and evasion I would be very interested to hear them. In the meantime, publishers: please don't block bots! Many of them are NOT evil and simply people trying to understand the scholarly landscape at scale.